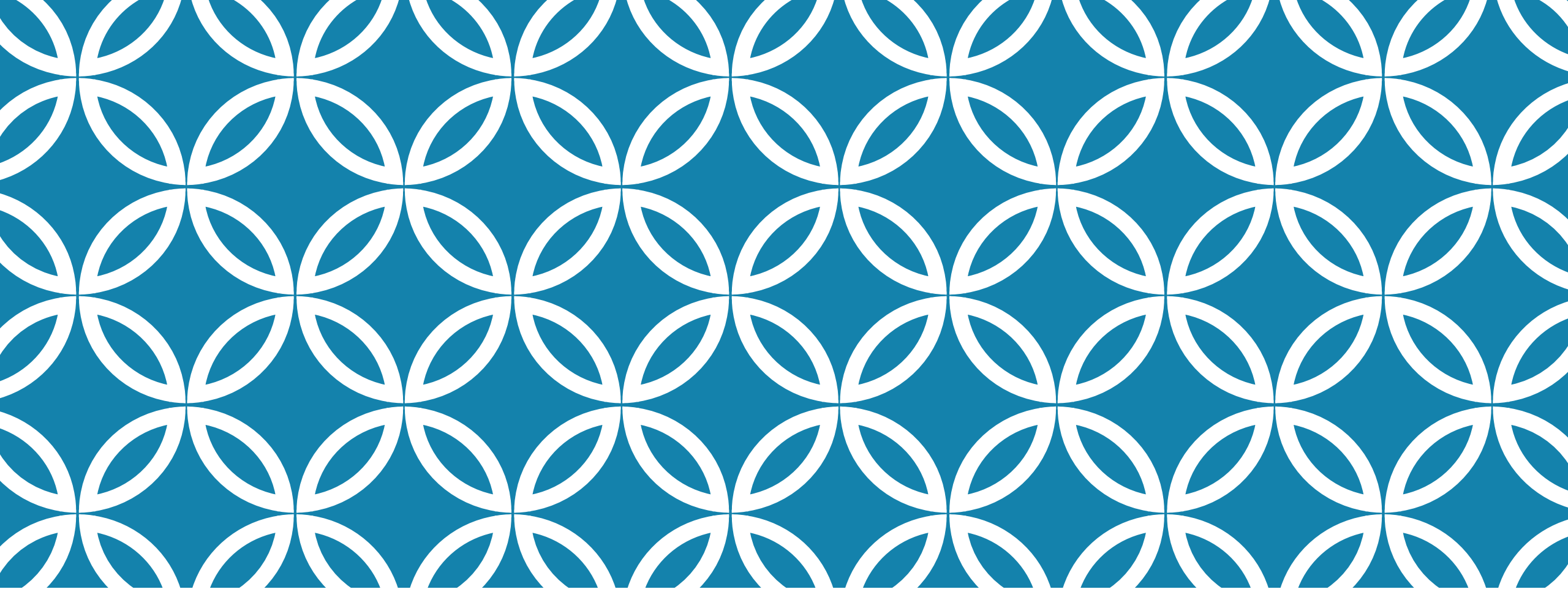




TECHNOLOGIES & SERVICES WEB

XML, SOAP, UDDI, CXF, REST et
les services web

M. Flavien SOMDA



INTRODUCTION AUX SERVICES WEB ET AUX SYSTÈMES DISTRIBUÉS

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Expliquer ce qu'est un système distribué
- Décrire l'évolution de la communication entre applications
- Définir les services web et leurs principales caractéristiques
- Distinguer les approches SOAP et REST
- Comprendre pourquoi REST est dominant dans les systèmes modernes
- Identifier les défis inhérents aux systèmes distribués

MOTIVATION : POURQUOI LES SERVICES WEB ?

- Les systèmes logiciels modernes existent rarement de manière isolée.
 - Des applications mobiles et plateformes cloud aux systèmes d'information d'entreprise et aux infrastructures IoT, les composants logiciels doivent communiquer, échanger des données et coordonner leurs actions à travers des réseaux.
 - Ce besoin d'interopérabilité a conduit à l'évolution des **systèmes distribués**, dans lesquels plusieurs composants autonomes collaborent pour atteindre un objectif commun.

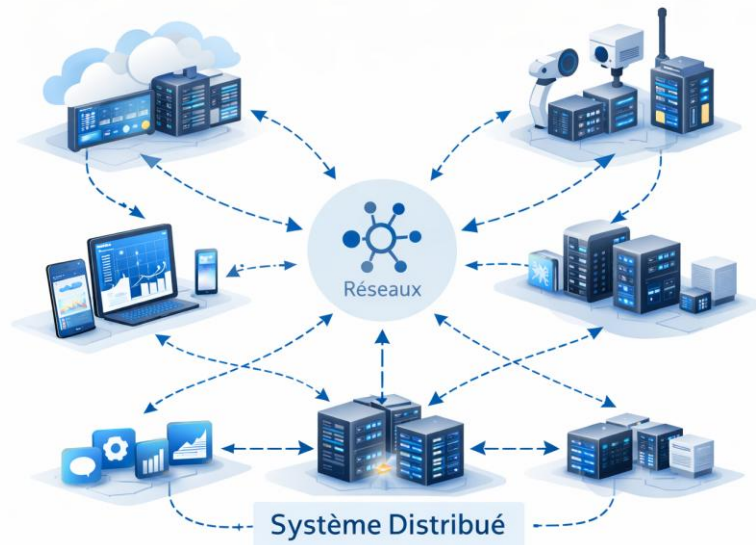
Les services web sont apparus comme une réponse à plusieurs défis fondamentaux :

- L'hétérogénéité des plateformes (différents langages de programmation, systèmes d'exploitation, matériels)
- La distribution géographique des systèmes
- Les frontières organisationnelles entre fournisseurs et consommateurs de services
- Le besoin de scalabilité, de maintenabilité et de réutilisation

DÉFINITION D'UN SYSTÈME DISTRIBUÉ

Un **système distribué** peut être défini comme suit :

Une collection d'ordinateurs indépendants qui apparaît à ses utilisateurs comme un système unique et cohérent.



Les principales caractéristiques d'un système distribué incluent :

Concurrence : plusieurs composants fonctionnent simultanément.

Absence de mémoire partagée : la communication se fait par échange de messages.

Défaillances partielles : certains composants peuvent tomber en panne tandis que d'autres continuent de fonctionner.

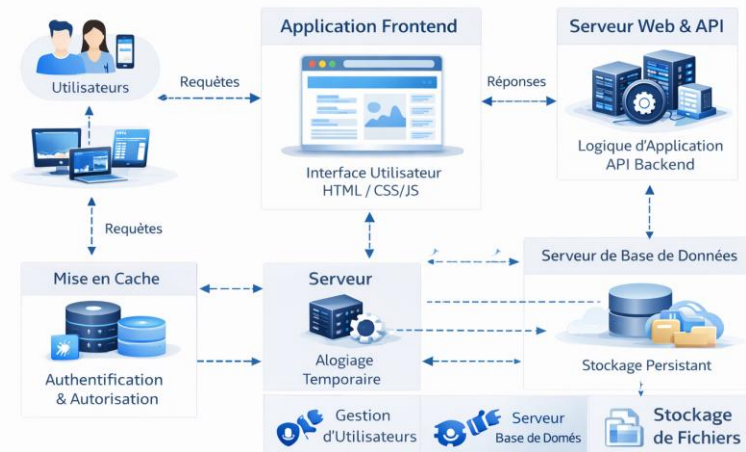
Absence d'horloge globale : la synchronisation temporelle est imparfaite.

DES EXEMPLES DE SYSTÈMES DISTRIBUÉS

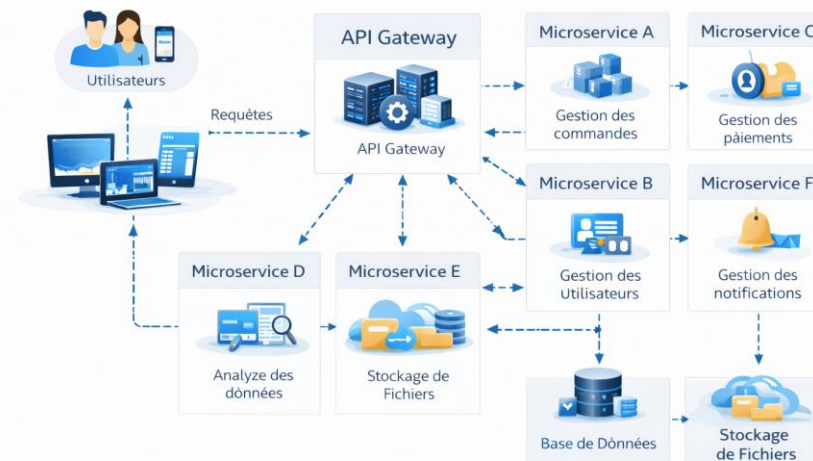
Des exemples de systèmes distribués incluent :

- Les applications web
- Les plateformes de cloud computing
- Les architectures microservices
- Les systèmes pair-à-pair (peer to peer)

Architecture d'une Application Web



Architecture Microservices



ÉVOLUTION DE LA COMMUNICATION ENTRE APPLICATIONS (1/2)

Applications monolithiques

Les premiers systèmes logiciels étaient souvent **monolithiques**, ce qui signifie que :

- Tous les composants étaient déployés ensemble
- La communication interne utilisait des appels de fonctions ou de procédures
- La montée en charge nécessitait la duplication de l'ensemble de l'application
- Bien que simples à concevoir au départ, les monolithes souffrent de :
 - Scalabilité limitée
 - Maintenance difficile
 - Couplage fort entre les composants

Architecture client-serveur

Le modèle client-serveur a introduit une séparation des responsabilités :

Client : interface utilisateur et logique d'interaction

Serveur : logique métier et gestion des données

La communication s'effectuait via des réseaux, généralement à l'aide de protocoles propriétaires. Cela a amélioré la scalabilité, mais a introduit des problèmes d'interopérabilité.

Middleware et RPC

Les systèmes de **Remote Procedure Call (RPC)** permettaient à un programme d'invoquer des procédures sur une machine distante comme si elles étaient locales. Bien que pratiques, les systèmes basés sur RPC :

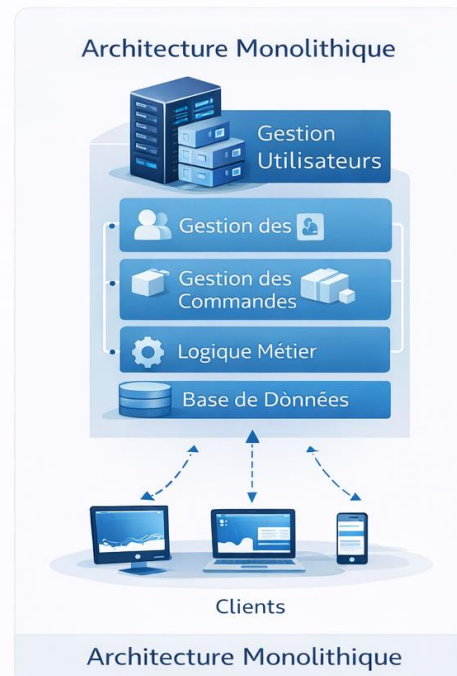
- Masquent les défaillances réseau
- Encouragent un couplage fort
- Sont difficiles à faire évoluer à l'échelle d'Internet

Ces limitations ont ouvert la voie à l'émergence des services web.

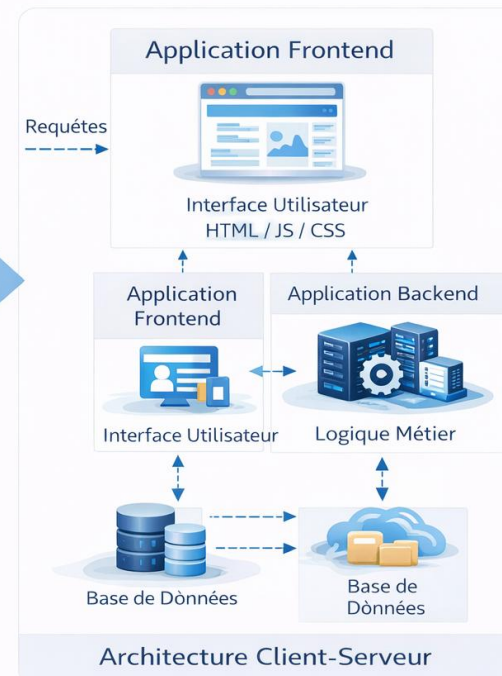
ÉVOLUTION DE LA COMMUNICATION ENTRE APPLICATIONS (2/2)

Applications monolithiques

Tous les composants sont déployés ensemble sur une même machine



Passage d'une Architecture Monolithique à une Architecture Client-Serveur



Applications client-serveur

Le frontend et le backend sont déployés sur des machines différentes et communiquent via le réseau

QU'EST-CE QU'UN SERVICE WEB ?

Un **service web** est un système logiciel conçu pour supporter des interactions interopérables entre machines via un réseau.

Les caractéristiques principales d'un web service sont :

- Utilisation de protocoles web standards
- Indépendance vis-à-vis des plateformes et des langages
- Accessibilité via un réseau
- Exposition d'une interface bien définie

Les services web reposent généralement sur :

- **HTTP** pour la communication
- Des formats de données standards (XML, JSON)
- Des **URI** pour l'identification

TYPES DE SERVICES WEB

Services web basés sur SOAP

SOAP (*Simple Object Access Protocol*) est une spécification de protocole qui définit :

- La structure des messages (basée sur XML)
- Des contrats stricts (WSDL)
- Un typage fort et une validation formelle

Avantages :

- Forte standardisation
- Support intégré pour la sécurité et les transactions

Inconvénients :

- Complexité élevée
- Messages verbeux
- Adaptation limitée aux systèmes à grande échelle sur le Web

Services web RESTful

REST (*Representational State Transfer*) est un **style architectural**, et non un protocole.

Caractéristiques clés :

- Construit au-dessus de HTTP
- Utilise les méthodes HTTP standards (GET, POST, PUT, DELETE)
- Considère toute entité comme une ressource
- Met l'accent sur des interactions sans état (*stateless*)

Les services RESTful sont :

- Légers
- Scalables
- Lisibles par l'humain
- Bien alignés avec les principes du Web

Ce cours se concentre principalement sur les **services web RESTful**.

LE RÔLE DU WEB ET DE HTTP

Le **World Wide Web** est le plus grand système distribué jamais construit. REST tire directement ses principes du fonctionnement du Web.

HTTP comme protocole applicatif

HTTP n'est pas simplement un protocole de transport. Il définit :

- La sémantique requête/réponse
- Des méthodes ayant une signification bien définie
- Des codes de statut décrivant les résultats
- Des en-têtes pour les métadonnées et le contrôle

Les services RESTful exploitent la **sémantique de HTTP**, plutôt que de réinventer des mécanismes de communication.

INTEROPÉRABILITÉ ET COUPLAGE FAIBLE

REST favorise fortement le couplage faible, ce qui le rend idéal pour les systèmes à grande échelle.

L'un des objectifs majeurs des services web est l'**interopérabilité**.

L'interopérabilité signifie que :

- Un client Java peut consommer un service écrit en Python
- Une application mobile peut interagir avec un backend cloud
- Les systèmes peuvent évoluer indépendamment

Le **couplage faible** est obtenu en :

- Évitant le partage d'état
- Utilisant des interfaces standardisées
- S'appuyant sur des messages auto-descriptifs

COMMUNICATION SYNCHRONE ET ASYNCHRONE

Communication synchrone

- Le client envoie une requête et attend une réponse
- Simple à implémenter
- Peut entraîner des blocages et une scalabilité réduite

Communication asynchrone

- Le client envoie une requête et poursuit son exécution
- La réponse peut arriver plus tard ou via un autre canal
- Améliore la scalabilité et la résilience

REST peut être combiné à des modèles asynchrones à l'aide de :

- Callbacks: une **fonction fournie à l'avance** qui sera **appelée automatiquement plus tard** lorsqu'un événement se produit ou qu'un traitement est terminé (**Exemple** : après le chargement de données, on appelle une fonction pour mettre à jour l'interface.)
- Webhooks: un **appel HTTP automatique** envoyé par une application **vers une autre application** lorsqu'un événement survient (**Exemple** : un service de paiement envoie une requête HTTP à votre serveur quand un paiement est validé.)
- Files de messages: un système **dépose un message** dans une file, et un autre système le **consomme plus tard**, à son propre rythme

HTTP est principalement synchrone, bien que les frameworks modernes atténuent cela grâce aux E/S non bloquantes.

DÉFIS DES SYSTÈMES DISTRIBUÉS

La conception de services web nécessite la compréhension de défis fondamentaux :

- **Latence réseau** : la communication est lente et imprévisible
- **Défaillances partielles** : certains services peuvent être indisponibles
- **Menaces de sécurité** : les données transitent sur des réseaux publics
- **Gestion des versions** : les services évoluent dans le temps
- **Scalabilité** : la charge peut augmenter de manière drastique

REST répond à nombre de ces défis grâce à :

- L'absence d'état
- La mise en cache
- Des interfaces uniformes
- Une architecture en couches



FONDAMENTAUX DE HTTP ET DE L'ARCHITECTURE DU WEB

Ce chapitre établit les bases de HTTP et de l'architecture du Web, indispensables à la compréhension de REST.

POURQUOI HTTP EST FONDAMENTAL POUR REST

Les services web RESTful ne sont pas construits sur HTTP par hasard ; ils sont fondamentalement **façonnés par HTTP**. Pour concevoir correctement des API RESTful, il est indispensable de comprendre HTTP non seulement comme un protocole de communication, mais comme un **protocole applicatif doté d'une sémantique précise**.

❌ Une erreur fréquente

Considérer **HTTP** comme une simple **couche de transport**

L'utiliser comme on utiliserait des **sockets TCP**

Réimplémenter toute la **logique applicative** dans le corps des requêtes

👉 Cette approche **détourne HTTP de son rôle initial**

🚫 Ce que REST rejette

REST **refuse** l'usage de HTTP comme un simple tunnel

La logique métier ne doit **pas ignorer le protocole**

HTTP ne doit pas être réduit à un simple canal de données

✅ L'approche recommandée par REST

REST encourage à utiliser **HTTP tel qu'il a été conçu** :

Méthodes HTTP : GET, POST, PUT, DELETE, ...

Codes de statut : 200, 201, 400, 404, 500, ...

En-têtes HTTP : authentication, contenu, cache, etc.

Mécanismes de cache : performance et scalabilité

LE WEB COMME SYSTÈME DISTRIBUÉ

Le World Wide Web est un **système distribué à grande échelle**, ouvert, caractérisé par :

- Des milliards de clients et de serveurs
- Des interactions sans état
- Une cohérence faible
- Des exigences extrêmes de scalabilité
- Une forte tolérance aux pannes

Contrairement aux systèmes distribués traditionnels, conçus dans des environnements contrôlés, le Web fonctionne dans des conditions **hautement imprévisibles** :

- Les réseaux peuvent tomber en panne
- Les clients peuvent disparaître
- Les serveurs peuvent être surchargés
- La latence est imprévisible

HTTP a été conçu en tenant compte de ces réalités.

REST adopte ces mêmes contraintes afin d'assurer la scalabilité et la robustesse.

PRÉSENTATION GÉNÉRALE DU PROTOCOLE HTTP

Position dans la pile réseau

HTTP opère au niveau **applicatif** du modèle TCP/IP :

- Application : HTTP
- Transport : TCP
- Réseau : IP

Bien que HTTP repose sur TCP pour le transport fiable des données, il définit **sa propre sémantique applicative**, essentielle à la conception REST.

Caractère sans état de HTTP

HTTP est un protocole **sans état** (*stateless*) :

- Chaque requête est indépendante
- Le serveur ne conserve pas le contexte du client
- Toutes les informations nécessaires doivent être incluses dans la requête

Cette propriété :

- Simplifie la conception des serveurs
- Améliore la scalabilité
- Facilite la répartition de charge
- Réduit le couplage

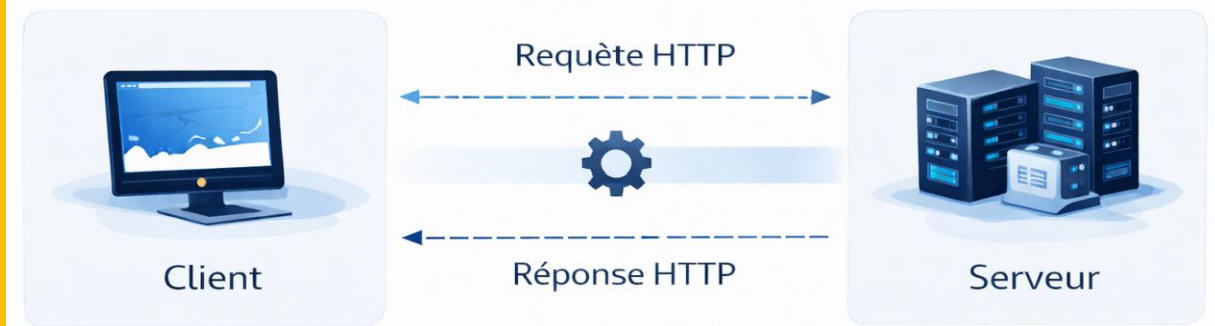
REST adopte pleinement cette caractéristique et en fait une contrainte architecturale centrale.

MODÈLE REQUÊTE—RÉPONSE DE HTTP

La communication HTTP suit un modèle **requête—réponse** :

1. Le client envoie une requête HTTP
2. Le serveur traite la requête
3. Le serveur renvoie une réponse HTTP

Ce modèle simple est à la base de toutes les interactions RESTful.



Structure d'une requête HTTP

Une requête HTTP est composée de :

Ligne de requête

- Méthode
- URI
- Version HTTP

En-têtes

- Métadonnées sur la requête
- Informations de contrôle

Corps (optionnel)

- Utilisé notamment par POST ou PUT
- Contient une représentation de ressource

Structure d'une réponse HTTP

Une réponse HTTP contient :

Ligne de statut

- Version HTTP
- Code de statut
- Message associé

En-têtes

- Métadonnées sur la réponse

Corps (optionnel)

- Représentation de la ressource

MÉTHODES HTTP (VERBES)

HTTP définit un ensemble restreint de méthodes, chacune ayant une **signification précise**. REST repose fortement sur cette sémantique.

Les principales méthodes sont :

- GET
- POST
- PUT
- PATCH
- DELETE

Les propriétés de **sécurité** et **d'idempotence** sont fondamentales pour la robustesse des systèmes distribués.

CODES DE STATUT HTTP

Les codes de statut indiquent le **résultat d'une requête**.

Catégories

1xx : Information

2xx : Succès

3xx : Redirection

4xx : Erreur client

5xx : Erreur serveur

Un usage correct des codes de statut est un indicateur clé de la qualité d'une API REST.

EN-TÊTES HTTP

Les en-têtes permettent l'échange de **métadonnées**.

Ils sont essentiels pour :

- La négociation de contenu
- La sécurité
- La mise en cache
- Le contrôle des requêtes et réponses

URI ET IDENTIFICATION DES RESSOURCES

Chaque ressource REST est identifiée par une **URI unique**.

Bonnes pratiques :

- URI stables
- URI descriptives
- Utilisation de noms, pas de verbes

MISE EN CACHE HTTP

HTTP intègre des mécanismes puissants de mise en cache :

- Cache-Control
- Expires
- Etag
- Last-Modified

REST encourage fortement l'utilisation de ces mécanismes pour améliorer les performances.



STYLE ARCHITECTURAL REST : PRINCIPES ET CONTRAINTES

Ce chapitre présente REST comme un style architectural rigoureux reposant sur des contraintes précises.

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

Définir REST avec précision

Expliquer chacune des contraintes REST

Identifier les violations de REST

Comprendre le rôle de HATEOAS

Évaluer si une API est réellement RESTful

QU'EST-CE QUE REST ?

- REST, acronyme de *Representational State Transfer*, est un **style architectural** introduit par **Roy T. Fielding** dans sa thèse de doctorat en 2000.
- Contrairement à des protocoles tels que HTTP ou SOAP, REST n'est ni une norme ni une spécification.
- Il définit plutôt un **ensemble de contraintes architecturales** qui guident la conception de systèmes distribués scalables, maintenables et évolutifs.

Un style architectural décrit :

- Un ensemble de contraintes de conception
- Les rôles des composants
- Les interactions entre les composants
- Les propriétés induites par ces contraintes

LES SIX CONTRAINTES DE REST

REST est défini par **six contraintes architecturales**. Chaque contrainte contribue à des propriétés spécifiques du système. Supprimer ou violer une contrainte affaiblit ces propriétés.

REST repose sur six contraintes :

1. Client—serveur
2. Sans état (*stateless*)
3. Cacheable
4. Interface uniforme
5. Système en couches
6. Code à la demande (Code-on-Demand, optionnelle)

CONTRAINTE CLIENT—SERVEUR

Définition

La contrainte client—serveur impose une séparation claire entre :

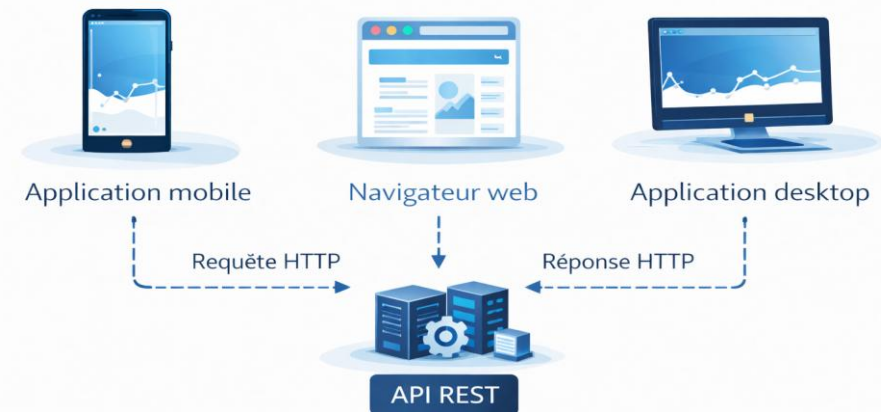
- **Les clients**, responsables de l'interaction avec l'utilisateur
- **Les serveurs**, responsables du stockage des données et de la logique métier

Avantages

Cette séparation :

- Améliore la portabilité des clients
- Permet aux serveurs de monter en charge indépendamment
- Simplifie l'évolution du système
- Différents clients peuvent tous consommer la même API REST.

Une application mobile, un navigateur web et une application desktop peuvent tous consommer la même API REST



CONTRAINTE SANS ÉTAT (STATELESS)

Définition

Dans REST, chaque requête envoyée par un client doit contenir toutes les informations nécessaires pour que le serveur puisse la comprendre et la traiter. Le serveur ne doit pas conserver d'état de session entre les requêtes.

Conséquences

- Le serveur ne se souvient pas des interactions précédentes
- Les informations d'authentification doivent être envoyées avec chaque requête

L'état est conservé côté client, et non côté serveur

Avantages

L'absence d'état :

- Améliore la scalabilité
- Simplifie la répartition de charge
- Renforce la fiabilité
- Réduit l'utilisation de la mémoire serveur

Violations courantes

- Sessions HTTP stockées côté serveur
- Authentification avec état sans utilisation de jetons
- Dépendances implicites à l'ordre des requêtes

CONTRAINTE DE MISE EN CACHE

Définition

Les réponses doivent indiquer explicitement si elles sont **cacheables** ou non.

La mise en cache permet :

- Aux clients ou aux intermédiaires de réutiliser les réponses
- De réduire le trafic réseau
- De diminuer la charge sur les serveurs

Support HTTP pour la mise en cache

REST exploite les mécanismes HTTP tels que :

- Cache-Control
- Etag
- Expires
- Last-Modified

Impact sur les performances

Une mise en cache efficace :

- Améliore la latence
- Renforce la scalabilité
- Permet des systèmes à haut débit

Une mise en cache mal conçue peut toutefois entraîner des données obsolètes ou incohérentes.

CONTRAINTE D'INTERFACE UNIFORME (1/3)

L'interface uniforme est la contrainte la plus importante et la plus distinctive de REST. C'est elle qui différencie REST des autres styles architecturaux.

Identification des ressources

Les ressources sont identifiées par des **URI**.

Principes clés :

- Les ressources sont des noms, pas des verbes
- Les URI sont stables et significatives
- Les détails d'implémentation internes sont masqués

Illustration

`/students/42`

Et non

~~`/getStudent?id=42`~~

CONTRAINTE D'INTERFACE UNIFORME (2/3)

Manipulation des ressources via des représentations

Les clients interagissent avec les ressources en échangeant des **représentations**.

Une représentation :

- Décrit l'état de la ressource
- Est indépendante du modèle interne du serveur
- Peut exister sous plusieurs formats (JSON, XML, HTML)

Les clients modifient l'état côté serveur en envoyant des représentations mises à jour.

Messages auto-descriptifs

Chaque message doit contenir suffisamment d'informations pour être compris de manière isolée.

Cela inclut :

- La méthode HTTP
- Les en-têtes
- Le type de média
- Le code de statut

Les messages auto-descriptifs :

- Améliorent la visibilité
- Permettent l'intervention d'intermédiaires (proxies, passerelles)
- Réduisent le couplage

CONTRAINTE D'INTERFACE UNIFORME (3/3)

Hypermedia comme moteur de l'état applicatif (HATEOAS)

HATEOAS est la contrainte REST la plus mal comprise.

Elle signifie que :

- Les clients découvrent dynamiquement les actions disponibles
- Le serveur fournit des liens vers les ressources associées
- Les clients ne codent pas en dur les workflows

```
{  
  "id": 42,  
  "name": "Alice",  
  "links": [  
    { "rel": "courses", "href": "/students/42/courses" },  
    { "rel": "delete", "href": "/students/42", "method": "DELETE" }  
  ]  
}
```

HATEOAS améliore l'évolutivité, réduit le couplage client-serveur et permet des workflows flexibles

CONTRAINTE DE SYSTÈME EN COUCHES

Définition

Un système en couches autorise la présence d'intermédiaires entre le client et le serveur :

Proxies

Passerelles (*gateways*)

Répartiteurs de charge

Caches

Les clients n'ont pas besoin de savoir s'ils communiquent directement avec le serveur final.

Avantages

- Amélioration de la scalabilité
- Renforcement de la sécurité
- Simplification de l'évolution du système

Exemples :

- API gateways
- Équilibreur de charge (load balancer)
- Reverse proxies
- Couches CDN

Le client communique avec une API exposée derrière un équilibreur de charge



CONTRAINTE DE CODE À LA DEMANDE (OPTIONNEL)

Définition

Les serveurs peuvent, de manière optionnelle, envoyer du **code exécutable** aux clients.

Exemples :

JavaScript dans les applications web

Scripts de validation côté client

Caractéristiques

- Améliore la flexibilité
- Réduit la complexité côté client
- Réduit la visibilité (raison pour laquelle cette contrainte est optionnelle)

Les systèmes REST peuvent être pleinement RESTful sans cette contrainte.

PROPRIÉTÉS INDUITES PAR REST

Lorsque toutes les contraintes sont respectées, REST permet :

- Scalabilité
- Simplicité
- Visibilité
- Fiabilité
- Performance
- Évolutivité

REST ET MICROSERVICES

REST est couramment utilisé dans les architectures microservices, mais :

- REST $\neq$ microservices
- Les microservices peuvent utiliser d'autres styles de communication

REST est particulièrement adapté aux :

- API externes
- Services publics
- Systèmes orientés clients

IDÉES REÇUES COURANTES SUR REST

Ci-dessous quelques idées reçues erronées sur REST:

- REST, c'est juste du JSON sur HTTP ✗
- REST impose uniquement le CRUD ✗
- HATEOAS est optionnel ✗
- REST est facile ✗

Un REST authentique exige **rigueur, discipline et réflexion architecturale**.



MODÉLISATION DES RESSOURCES ET CONCEPTION DES URI

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Identifier les ressources dans un domaine
- Distinguer ressources et représentations
- Concevoir des URI claires et significatives
- Modéliser correctement les relations
- Éviter les anti-patterns REST courants

POURQUOI LA MODÉLISATION DES RESSOURCES EST CRUCIALE EN REST

Au cœur de REST se trouve une idée simple mais puissante : **tout est une ressource**. Bien que cette affirmation paraisse évidente, elle est à l'origine de nombreuses erreurs de conception dans les API dites RESTful. Une mauvaise modélisation des ressources conduit à :

- Des API confuses
- Un couplage fort entre client et serveur
- Des difficultés à faire évoluer le système
- Une mauvaise utilisation des méthodes HTTP

La modélisation des ressources est donc **l'activité de conception la plus importante** lors de la construction de services web RESTful. Contrairement à la conception orientée objet, qui se concentre sur les classes et les méthodes, REST se concentre sur les **ressources et leurs relations**.

QU'EST-CE QU'UNE RESSOURCE ?

Définition

Une **ressource** est tout concept qui peut être nommé, identifié, représenté, manipulé

Exemples de ressources :

- Un étudiant
- Un cours
- Une inscription
- Un document
- Un paiement
- Une collection d'entités

Contre-exemples:

Une ressource **n'est pas** :

- Une table de base de données
- Une classe dans le code
- Une fonction ou une méthode

Les ressources représentent des **entités conceptuelles**, et non des détails d'implémentation.

RESSOURCE VS REPRÉSENTATION

Il est essentiel de distinguer :

Ressource : le concept abstrait

Représentation : un instantané concret de son état

Par exemple :

Ressource : Étudiant n°42

Représentation : document JSON décrivant l'étudiant

```
{  
  "id": 42,  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

Une même ressource peut avoir plusieurs représentations :

- JSON
- XML
- HTML

Les API REST manipulent des **représentations**, et non la ressource elle-même.

IDENTIFICATION DES RESSOURCES DANS UN DOMAINE

Analyse du domaine

L'identification des ressources commence par l'**analyse du domaine**.

Sources typiques :

- Exigences métier
- Cas d'utilisation
- User stories
- Vocabulaire du domaine

Les **noms** présents dans le domaine indiquent souvent des ressources potentielles.

Exemple de domaine : système de gestion universitaire

Ressources potentielles :

- Étudiant
- Cours
- Enseignant
- Inscription
- Note

Ressources primaires et secondaires

Ressources primaires : entités centrales du domaine

Exemples : /students, /courses

Ressources secondaires (sous-ressources)

: existent en relation avec d'autres

Exemple : /students/42/courses

Les sous-ressources expriment des **relations**, et non une hiérarchie de stockage.

COLLECTIONS ET RESSOURCES INDIVIDUELLES

Collections

Une collection représente un groupe de ressources.

Exemple :

/students

Opérations :

GET /students → lister les étudiants

POST /students → créer un nouvel étudiant

Ressources individuelles

Une ressource individuelle est identifiée par une URI unique.

Exemple :

/students/42

Opérations :

GET → récupérer

PUT → remplacer

PATCH → mettre à jour partiellement

DELETE → supprimer

CONCEPTION DES URI : PRINCIPES FONDAMENTAUX

Utiliser des noms, pas des verbes

Les URI doivent représenter des **entités**, pas des actions.

Bon exemple :

`/students/42`

Mauvais exemples :

`/getStudent`

`/deleteStudent`

Les actions sont exprimées via les **méthodes HTTP**, et non via les URI.

Utiliser des noms au pluriel pour les collections

La cohérence est essentielle.

Recommandé :

`/students`

`/courses`

Éviter de mélanger les styles.

Conserver des URI stables

Les URI sont des **contrats** avec les clients.

Ne pas exposer :

- La structure de la base de données
- Les extensions de fichiers
- Les détails internes d'implémentation

Éviter :

`/api/v1/mysql/student_table`

URI HIÉRARCHIQUES VS RELATIONNELLES

URI hiérarchiques

Les hiérarchies expriment une relation forte ou une appartenance.

Exemple :

`/students/42/courses`

Signification :

Les cours associés à l'étudiant 42

Éviter l'imbrication excessive

Une imbrication trop profonde entraîne :

- Des URI longues
- Un couplage fort
- Une flexibilité réduite

À éviter :

`/universities/1/faculties/2/departments/3/students/42`

Préférer :

`/students/42`

Les relations peuvent être parcourues à l'aide de **liens**.

PARAMÈTRES DE CHEMIN VS PARAMÈTRES DE REQUÊTE

Paramètres de chemin

Utilisés pour identifier les ressources.

Exemple :

`/students/42`

Paramètres de requête

Utilisés pour :

- Filtrer
- Trier
- Paginer
- Rechercher

Exemple :

`/students?year=2024&sort=name&page=2`

Les paramètres de requête ne doivent **pas** servir à définir l'identité d'une ressource.

MODÉLISATION DES ACTIONS EN REST

REST décourage les URI orientées action, mais certaines actions sont inévitables.

Privilégier les transitions d'état

Au lieu de :

`/approveStudent`

Utiliser :

`PUT /students/42`

Avec une représentation reflétant le nouvel état.

Ressources d'action

Lorsque nécessaire, modéliser les actions comme des **ressources**.

Exemple :

`POST /students/42/suspensions`

Cela représente un événement de suspension comme une ressource.

IDEMPOTENCE ET CONCEPTION DES RESSOURCES

La modélisation des ressources doit respecter l'**idempotence** :

- PUT et DELETE doivent être idempotents
- POST n'est pas idempotent

Les ressources doivent être conçues de manière à ce que les requêtes répétées aient un comportement prévisible.

ANTI-PATTERNS COURANTS DANS LA CONCEPTION DES URI

Les pratiques suivantes dégradent la qualité et la pérennité des API et doivent donc être évitées:

- Verbes dans les URI
- Pluralisation incohérente
- Fuite des clés de base de données
- Encodage de la logique métier dans les chemins
- Imbrication excessive

EXEMPLE PRATIQUE : CONCEPTION D'UNE API UNIVERSITAIRE

Ressources :

/students

/students/{id}

/courses

/courses/{id}

/enrollments

Relations :

/students/{id}/courses

/courses/{id}/students

Filtrage :

/students?department=sea



MÉTHODES HTTP, CODES DE STATUT ET IDEMPOTENCE

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Choisir correctement les méthodes HTTP
- Comprendre la sécurité et l'idempotence
- Utiliser les codes de statut HTTP de manière appropriée
- Concevoir une gestion robuste des erreurs
- Éviter les anti-patterns REST courants

POURQUOI LA SÉMANTIQUE HTTP EST ESSENTIELLE EN REST

L'une des idées fausses les plus répandues dans la conception des API REST est de croire que HTTP n'est qu'un protocole de transport servant à déplacer des charges utiles JSON entre un client et un serveur.

En réalité, HTTP est un **protocole applicatif** doté d'une sémantique précisément définie. Les systèmes RESTful s'appuient sur cette sémantique pour atteindre la scalabilité, la fiabilité et la clarté.

VUE D'ENSEMBLE DES MÉTHODES HTTP

HTTP définit un ensemble restreint et fixe de méthodes (également appelées verbes). Chaque méthode possède une signification spécifique, indépendante de la manière dont elle est implémentée côté serveur.

REST n'invente pas de nouvelles méthodes ; il **réutilise les méthodes HTTP exactement telles qu'elles sont définies**.

Les méthodes les plus importantes en REST sont :

- GET
- POST
- PUT
- PATCH
- DELETE
- HEAD
- OPTIONS

GET : RÉCUPÉRATION DES REPRÉSENTATIONS DE RESSOURCES

Sémantique de GET

GET est utilisé pour récupérer une **représentation** d'une ressource.

Propriétés :

- **Sûre (safe)** : ne doit pas modifier l'état du serveur
- **Idempotente** : répéter la requête produit le même effet
- **Cacheable** : les réponses peuvent être mises en cache

Exemple :

GET /students/42

Ce que GET ne doit JAMAIS faire

Une requête GET ne doit jamais :

- Créer une ressource
- Mettre à jour une ressource
- Déclencher une logique métier avec des effets de bord

La violation de cette règle casse :

- Les mécanismes de mise en cache
- Les proxies
- Les moteurs de recherche
- Les hypothèses de sécurité

GET sur les collections

Exemple :

GET /students

Retourne :

- Une liste d'étudiants
- Éventuellement paginée
- Éventuellement filtrée via des paramètres de requête

POST : CRÉATION ET TRAITEMENT DE RESSOURCES

Sémantique de POST

POST est utilisé pour :

- Créer une nouvelle ressource au sein d'une collection
- Déclencher un traitement côté serveur
- Soumettre des données ne correspondant pas aux autres méthodes

Exemple :

POST /students

Caractéristiques de POST

Non idempotente

Non sûre

La réponse inclut souvent :

- 201 Created
- Un en-tête Location pointant vers la nouvelle ressource

Exemple de réponse :

HTTP/1.1 201 Created

Location: /students/42

POST vs PUT (aperçu)

POST est utilisé lorsque :

- Le serveur attribue l'identifiant de la ressource
- L'opération n'est pas idempotente

PUT est utilisé lorsque :

- Le client contrôle l'URI de la ressource
- L'opération est idempotente

PUT : REMPLACEMENT DES RESSOURCES

Sémantique de PUT

PUT remplace **entièrement** la représentation d'une ressource.

Exemple :

PUT /students/42

Idempotence de PUT

PUT est **idempotente** :

- Envoyer plusieurs fois la même requête PUT produit le même état final de la ressource

Cette propriété est cruciale pour :

- Les mécanismes de reprise (*retry*)
- La gestion des pannes réseau

PUT et création de ressources

PUT peut créer une ressource si elle n'existe pas, à condition que le client fournisse l'URI.

Exemple :

PUT /students/42

Ce comportement doit être **clairement documenté**.

PATCH : MISES À JOUR PARTIELLES

Motivation de PATCH

PUT remplace la ressource entière, ce qui peut être inefficace ou risqué.

PATCH permet de modifier **partiellement** une ressource.

Exemple :

PATCH /students/42

Sémantique de PATCH

- Pas nécessairement idempotente
- Applique un ensemble de modifications à une ressource
- Nécessite une définition précise du format de patch

Formats courants :

- JSON Patch
- JSON Merge Patch

Quand utiliser PATCH

Utiliser PATCH lorsque :

- Seule une partie de la ressource change
- Un remplacement complet est indésirable
- L'efficacité réseau est importante

DELETE : SUPPRESSION DES RESSOURCES

Sémantique de DELETE

DELETE supprime une ressource.

Exemple :

DELETE /students/42

Idempotence de DELETE

DELETE est **idempotent** :

- Supprimer une ressource déjà supprimée n'a pas d'effet supplémentaire

Réponses possibles :

- 204 No Content
- 404 Not Found (selon le choix de conception)

HEAD ET OPTIONS

HEAD

HEAD est identique à GET, sauf que :

- Aucun corps de réponse n'est retourné

Utilisée pour :

- Vérifier l'existence d'une ressource
- Inspecter les métadonnées
- Valider la fraîcheur du cache

OPTIONS

OPTIONS retourne :

- Les méthodes supportées
- Les options de communication

Utile pour :

- La découvrabilité de l'API
- Les requêtes *preflight* CORS: **requête HTTP automatique** (méthode OPTIONS) envoyée par le navigateur **avant la requête réelle**, pour vérifier si le serveur **autorise une requête cross-origin**.

MÉTHODES SÎRES ET IDEMPOTENTES

Méthodes sîres

Une méthode est dite **sîre** si elle ne modifie pas l'état du serveur.

Méthodes sîres :

- GET
- HEAD
- OPTIONS

Méthodes idempotentes

Une méthode est **idempotente** si sa répétition produit le même effet.

Méthodes idempotentes :

- GET
- PUT
- DELETE
- HEAD
- OPTIONS

POST et PATCH ne sont pas garanties idempotentes.

CODES DE STATUT HTTP EN REST

Les codes de statut communiquent le résultat d'une opération de manière claire et non ambiguë.

Codes de succès (2xx)

200 OK : requête réussie

201 Created : ressource créée

202 Accepted : requête acceptée pour traitement

204 No Content : succès sans corps de réponse

Codes d'erreur client (4xx)

400 Bad Request : entrée invalide

401 Unauthorized : authentification requise

403 Forbidden : accès refusé

404 Not Found : ressource inexistante

409 Conflict : conflit d'état de la ressource

422 Unprocessable Entity : erreur de validation sémantique

Codes d'erreur serveur (5xx)

500 Internal Server Error

502 Bad Gateway

503 Service Unavailable

Les codes 5xx indiquent des défaillances côté serveur, et non des erreurs client.

CONCEPTION DES RÉPONSES D'ERREUR

Une bonne API REST :

- Utilise des codes de statut appropriés
- Fournit des messages d'erreur significatifs
- Ne divulgue pas d'informations sensibles

Exemple de réponse d'erreur :

```
{  
  "error": "InvalidInput",  
  "message": "Email address is not valid"  
}
```

IDEMPOTENCE DANS LES SYSTÈMES DISTRIBUÉS

L'idempotence est essentielle car :

- Les réseaux sont peu fiables
- Les requêtes peuvent être rejouées
- Les clients peuvent expirer (*timeout*)

La conception d'opérations idempotentes :

- Empêche la création de ressources dupliquées
- Autorise des reprises sûres
- Améliore la résilience du système

ANTI-PATTERNS COURANTS

Éviter les anti-patterns courants suivants:

- Utiliser POST pour toutes les opérations
- Retourner 200 OK pour des erreurs
- Ignorer l'idempotence
- Encoder des actions dans les URI
- Surcharger GET avec des effets de bord

Ces pratiques violent les principes REST et la sémantique HTTP.



REPRÉSENTATION DES DONNÉES ET NÉGOCIATION DE CONTENU

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Expliquer la différence entre ressource et représentation
- Choisir des types de média appropriés
- Concevoir des représentations auto-descriptives
- Mettre en œuvre la négociation de contenu
- Utiliser efficacement l'hypermedia
- Supporter l'évolution des API en toute sécurité

POURQUOI LA REPRÉSENTATION DES DONNÉES EST ESSENTIELLE EN REST

Dans les services web RESTful, les clients et les serveurs n'échangent ni objets, ni lignes de base de données, ni structures de données internes. Ils échangent **des représentations de ressources**. Ces représentations constituent le **seul moyen** par lequel les clients peuvent observer et manipuler l'état côté serveur.

Des représentations mal conçues entraînent :

- Un couplage fort entre le client et le serveur
- Une évolution difficile de l'API
- Une utilisation inefficace du réseau
- Une sémantique ambiguë

À l'inverse, des représentations bien conçues permettent :

- L'interopérabilité entre plateformes et langages
- La compatibilité ascendante
- Le support de multiples clients ayant des besoins différents
- Une maintenabilité à long terme

RESSOURCE, ÉTAT ET REPRÉSENTATION

État de la ressource

Une ressource possède un **état**, c'est-à-dire l'ensemble des valeurs qui la décrivent à un instant donné.

Exemple (état conceptuel d'un étudiant) :

- Identifiant
- Nom
- Email
- Statut d'inscription

Cet état existe **côté serveur**.

Représentation de l'état

Une **représentation** est une forme sérialisée de l'état de la ressource, transférée sur le réseau.

Caractéristiques clés :

- Un instantané dans le temps
- Auto-descriptive
- Indépendante de l'implémentation serveur
- Encodée à l'aide d'un type de média

Les clients n'accèdent jamais directement à l'état de la ressource ; ils ne voient que ses représentations.

TYPES DE MÉDIA (TYPES MIME)

Définition

Un **type de média** (ou type MIME) identifie le format d'une représentation.

Forme générale :

type/sous-type

Exemples :

- application/json
- application/xml
- text/html

Les types de média sont fondamentaux en REST car ils permettent aux clients de comprendre comment interpréter une représentation.

Rôle des types de média en REST

Les types de média définissent :

- La syntaxe (structure des données)
- La sémantique (signification des champs)
- Les règles de traitement

En REST, les types de média font partie de l'**interface uniforme**, ce qui favorise le couplage faible.

FORMATS DE REPRÉSENTATION COURANTS

JSON (JavaScript Object Notation)

JSON est le format le plus largement utilisé dans les API REST.

Exemple :

```
{  
  "id": 42,  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

Avantages :

Léger

Lisible par l'humain

Support natif dans la plupart des langages

Facile à parser et à générer

Limites :

Pas de validation de schéma intégrée

Support limité des liens et métadonnées sans modélisation explicite

XML (eXtensible Markup Language)

XML a historiquement dominé les services web.

Exemple :

```
<student>  
  <id>42</id>  
  <name>Alice</name>  
  <email>alice@example.com</email>  
</student>
```

Avantages :

Support fort des schémas (XSD)

Espaces de noms

Outils matures

Inconvénients :

Verbeux

Moins populaire dans les API REST modernes

HTML comme représentation

HTML est une représentation REST valide, en particulier pour les applications web.

Exemples :

Les navigateurs sont des clients REST

Les pages web sont des représentations de ressources

Les liens sont des contrôles hypermédia

Les représentations HTML supportent naturellement **HATEOAS**.

REPRÉSENTATIONS AUTO-DESCRIPTIVES

Une représentation REST doit être **auto-descriptive**, c'est-à-dire contenir suffisamment d'informations pour être comprise sans connaissance externe.

Cela inclut :

- Le type de média
- L'encodage des caractères
- Les liens vers les ressources associées
- La signification sémantique des champs

Les messages auto-descriptifs améliorent :

- La visibilité
- Le débogage
- Le traitement par des intermédiaires

HYPERMEDIA DANS LES REPRÉSENTATIONS

Contrôles hypermédia

Les contrôles hypermédia sont des liens ou des actions intégrés dans les représentations.

Exemple :

```
{  
  "id": 42,  
  "name": "Alice",  
  "links": [  
    { "rel": "self", "href": "/students/42" },  
    { "rel": "courses", "href": "/students/42/courses" }  
  ]  
}
```

Ces liens indiquent au client :

Quelles ressources associées existent

Quelles actions sont possibles ensuite

Avantages de l'hypermedia

L'hypermedia :

- Réduit le couplage
- Permet l'évolution de l'API
- Favorise la découvrabilité
- Respecte les contraintes REST

Sans hypermedia, les clients doivent coder en dur les URI et les workflows.

NÉGOCIATION DE CONTENU : MOTIVATION

Différents clients peuvent avoir des besoins différents :

- Les navigateurs web préfèrent HTML
- Les applications mobiles préfèrent JSON
- Les systèmes hérités peuvent nécessiter XML

La négociation de contenu permet au client et au serveur de se mettre d'accord sur :

- Le format de représentation
- La langue
- L'encodage

Cette négociation s'effectue **dynamiquement à l'exécution.**

NÉGOCIATION DE CONTENU PILOTÉE PAR LE SERVEUR

L'en-tête Accept

Les clients expriment leurs préférences de types de média à l'aide de l'en-tête Accept.

Exemple :

Accept: application/json

Ou avec plusieurs préférences :

Accept: application/json,
application/xml;q=0.8

La valeur q indique le poids de préférence.

Comportement du serveur

Le serveur :

- Examine l'en-tête Accept
- Choisit la meilleure représentation disponible
- La retourne avec l'en-tête Content-Type approprié

Exemple d'en-tête de réponse :

Content-Type: application/json

NÉGOCIATION PILOTÉE PAR LE CLIENT ET APPROCHES HYBRIDES

Négociation pilotée par le client

Le client demande explicitement un format via :

- Des paramètres de requête
- Des suffixes d'URI

Exemple :

`/students/42?format=json`

Cette approche est simple, mais **moins RESTful**.

Approches hybrides

De nombreuses API combinent :

- Les en-têtes Accept
- Des paramètres de requête par commodité

Bien que pragmatique, cette approche doit être utilisée avec précaution afin d'éviter toute ambiguïté.

NÉGOCIATION DE LA LANGUE ET DE L'ENCODAGE

La négociation de contenu s'applique également à :

La langue (Accept-Language)

L'encodage des caractères (Accept-Charset)

Exemple :

Accept-Language: en, fr;q=0.9

Cela permet l'**internationalisation** des API REST.

CONCEPTION DES SCHÉMAS DE REPRÉSENTATION

Nommage des champs et sémantique

Bonnes pratiques :

- Utiliser des conventions de nommage cohérentes
- Éviter les abréviations
- Privilégier la clarté à la concision

Exemple :

firstName plutôt que fn

Champs optionnels et obligatoires

Les représentations doivent :

- Distinguer clairement les champs obligatoires
- Tolérer l'absence des champs optionnels
- Supporter la compatibilité ascendante

FAIRE ÉVOLUER LES REPRÉSENTATIONS EN TOUTE SÉCURITÉ

Les API évoluent dans le temps. La conception des représentations doit permettre cette évolution.

Changements sûrs :

- Ajout de nouveaux champs
- Ajout de nouveaux liens
- Support de nouveaux types de média

Changements cassants :

- Suppression de champs
- Modification de la signification d'un champ
- Changement de type de données

La négociation de contenu permet le versionnement par type de média, plus conforme à REST que le versionnement dans l'URI.

ANTI-PATTERNS COURANTS

Les anti-patterns suivants doivent être évités.

- Ignorer les types de média
- Toujours retourner du JSON sans négociation
- Utiliser une seule représentation pour tous les clients
- Coder les workflows en dur côté client
- Traiter les représentations comme de simples DTO (Data Transfer Object)

Ces pratiques réduisent la flexibilité et violent les principes REST.



BONNES PRATIQUES DE CONCEPTION DES API RESTFUL

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Appliquer les principes REST dans des conceptions d'API réelles
- Éviter les erreurs courantes de conception
- Justifier les choix de conception sur le plan architectural
- Évaluer la qualité d'une API RESTful
- Produire des conceptions d'API de niveau professionnel

DE LA THÉORIE À LA PRATIQUE

Après avoir étudié les principes REST, la sémantique HTTP, la modélisation des ressources et la représentation des données, nous sommes maintenant en mesure d'aborder une question critique :

Comment concevoir une API RESTful propre, robuste, scalable, sécurisée et évolutive ?

CONCEVOIR LES API AUTOUR DES RESSOURCES, PAS DES CAS D'USAGE

Raisonnement orienté ressources

Une erreur fréquente chez les débutants consiste à concevoir les API autour de cas d'usage ou d'actions :

/registerStudent

/assignCourse

/calculateAverage

Cette approche conduit à :

- Des URI basées sur des verbes
- Une surutilisation de POST
- Un couplage fort aux workflows métier

Les API RESTful doivent au contraire être conçues autour des **ressources** et de leur **état**.

Approche correcte :

/students

/courses

/enrollments

Modéliser les processus métier comme des ressources

Lorsqu'un processus métier existe, modéliser son résultat ou son artefact comme une ressource.

Exemple :

Au lieu de :

/approveEnrollment

Utiliser :

POST /enrollments

L'inscription elle-même devient une ressource possédant un cycle de vie.

UTILISER LES MÉTHODES HTTP CORRECTEMENT ET DE MANIÈRE COHÉRENTE

Une signification par méthode

Chaque méthode HTTP possède une signification sémantique unique. Il ne faut pas surcharger les méthodes.

Correspondance correcte :

- GET → récupérer
- POST → créer ou traiter
- PUT → remplacer
- PATCH → mise à jour partielle
- DELETE → supprimer

Pratiques incorrectes :

- Utiliser POST pour la lecture
- Utiliser GET pour déclencher des mises à jour
- Utiliser PUT pour des modifications partielles

Respecter la sécurité et l'idempotence

Concevoir les API de sorte que :

- GET ne modifie jamais l'état
- PUT et DELETE soient idempotentes
- POST ne soit utilisé que lorsque l'idempotence n'est pas requise

Cela garantit :

- Des reprises sûres (*retries*)
- Une mise en cache correcte
- Un comportement prévisible

UTILISER DES URI SIGNIFICATIVES ET COHÉRENTES

Conventions de nommage

Bonnes pratiques :

- Utiliser des minuscules
- Utiliser des tirets si nécessaire
- Utiliser des noms au pluriel pour les collections

Exemple :

`/students`

`/student-courses`

Éviter :

`/GetStudents`

`/studentCourses`

Éviter la fuite des détails d'implémentation

Les URI ne doivent pas exposer :

- Le langage de programmation
- La structure de la base de données
- Les extensions de fichiers
- Les noms internes de services

Mauvais :

`/api/v1/mysql/students.php`

Bon :

`/students`

UTILISER LES PARAMÈTRES DE REQUÊTE POUR LE FILTRAGE, LE TRI ET LA PAGINATION

Filtrage

Exemple :

GET /students?department=cs

Tri

Exemple :

GET /students?sort=name,-age

Pagination

Exemple :

GET
/students?page=2&size=20

Toujours documenter :

- Les valeurs par défaut
- La taille maximale de page
- Les règles de tri

CONCEVOIR DES REPRÉSENTATIONS CLAIRES ET COHÉRENTES

Garder des représentations ciblées

Les représentations doivent :

- Inclure uniquement les champs pertinents
- Éviter les imbrications inutiles
- Être cohérentes entre les endpoints

Ne pas exposer :

- Des identifiants internes inutilement
- Des champs spécifiques à la base de données
- Des métadonnées techniques

Utiliser l'hypermedia lorsque c'est approprié

Inclure des liens vers :

Les ressources associées

Les actions disponibles

Les états possibles suivants

Exemple :

```
{  
  "id": 42,  
  "name": "Alice",  
  "links": [  
    { "rel": "self", "href": "/students/42" },  
    { "rel": "enroll", "href": "/enrollments", "method": "POST" }  
  ]  
}
```

L'hypermedia améliore la découvrabilité et l'évolutivité.

LA GESTION DES ERREURS COMME PARTIE DU CONTRAT D'API

Utiliser des codes de statut HTTP appropriés

Les erreurs doivent être communiquées en utilisant :

Des codes de statut corrects

Des charges utiles d'erreur cohérentes

Exemple :

```
{  
  "error": "ValidationError",  
  "message": "Email address is invalid"  
}
```

Ne pas :

- Toujours retourner 200 OK
- Cacher les erreurs dans le corps de la réponse

Être explicite, mais sécurisé

Les messages d'erreur doivent :

- Être utiles aux clients
- Éviter de divulguer des informations sensibles
- Être cohérents à l'échelle de l'API

VERSIONNEMENT ET ÉVOLUTIVITÉ

Minimiser les changements cassants

REST encourage :

- La compatibilité ascendante
- Les changements additifs

Stratégies d'évolution sûres :

- Ajouter des champs
- Ajouter des liens
- Supporter de nouveaux types de média

Approches de versionnement

Approches courantes :

- Versionnement dans l'URI (/v1)
- Versionnement via les en-têtes
- Versionnement par type de média

Du point de vue REST, le versionnement par type de média est préférable, mais des choix pragmatiques peuvent s'appliquer.

SÉCURITÉ DÈS LA CONCEPTION

La conception d'une API RESTful doit intégrer la sécurité dès le départ.

Bonnes pratiques :

- Utiliser exclusivement HTTPS
- Ne jamais exposer de données sensibles
- Utiliser des mécanismes d'authentification standard
- Valider toutes les entrées

CONSIDÉRATIONS DE PERFORMANCE ET DE SCALABILITÉ

Une bonne conception REST améliore les performances en :

- Exploitant la mise en cache
- Réduisant la taille des charges utiles
- Évitant les API trop bavardes (*chatty*)
- Utilisant la pagination

Éviter :

- Des réponses fortement imbriquées
- La sur-collecte de données (*over-fetching*)
- Un nombre excessif d'allers-retours

LA DOCUMENTATION COMME ARTEFACT DE PREMIER ORDRE

Une API n'est réellement utilisable que si elle est bien documentée.

La documentation doit inclure :

- Descriptions des ressources
- Sémantique des méthodes
- Exemples de requêtes et de réponses
- Codes d'erreur
- Exigences d'authentification

La documentation n'est pas optionnelle ; elle fait partie du contrat d'API.

ANTI-PATTERNS COURANTS DES API REST

Anti-patterns courants des API REST à éviter:

- API de style RPC sur HTTP
- URI orientées action
- Ignorer les codes de statut HTTP
- Surcharger POST
- Retourner des représentations incohérentes
- Coder en dur les workflows côté client

Ces anti-patterns conduisent à des API fragiles et difficiles à maintenir.

LISTE DE VÉRIFICATION POUR LES API RESTFUL

Avant de publier une API, vérifier :

- Les ressources sont-elles clairement identifiées ?
- Les méthodes HTTP sont-elles utilisées correctement ?
- Les URI sont-elles stables et significatives ?
- Les représentations sont-elles auto-descriptives ?
- Les erreurs sont-elles gérées correctement ?
- L'API est-elle évolutive ?

Cette liste est particulièrement utile pour les examens et les revues de code.



GESTION DES ERREURS ET VERSIONNEMENT DES API

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Classer et gérer correctement les erreurs des API REST
- Utiliser les codes de statut HTTP de manière cohérente
- Concevoir des réponses d'erreur sécurisées et significatives
- Comprendre la compatibilité ascendante
- Choisir des stratégies de versionnement appropriées
- Gérer l'évolution des API de manière professionnelle

POURQUOI LES ERREURS ET L'ÉVOLUTION SONT CRUCIALES DANS LES API REST

Dans un monde idéal, les services web RESTful fonctionneraient toujours correctement et ne changeraient jamais. En réalité, les pannes sont inévitables et les systèmes logiciels doivent évoluer. Les réseaux échouent, les clients envoient des données invalides, les serveurs deviennent surchargés et les exigences métier évoluent avec le temps.

Une API REST bien conçue doit donc :

- Communiquer les erreurs de manière claire et cohérente
- Distinguer les erreurs côté client des défaillances côté serveur
- Éviter de casser les clients existants lors des évolutions
- Supporter des changements progressifs et contrôlés

La gestion des erreurs et le versionnement ne sont pas des fonctionnalités optionnelles. Ce sont des **aspects centraux de la qualité d'une API**, qui influencent fortement la maintenabilité, l'utilisabilité et la fiabilité.

LE RÔLE DE HTTP DANS LA GESTION DES ERREURS

REST n'invente pas son propre modèle d'erreur. Il s'appuie sur les **codes de statut HTTP** pour communiquer le résultat des requêtes.

Une règle fondamentale de la conception RESTful est :

Utiliser les codes de statut HTTP pour exprimer le résultat d'une opération, et non des codes d'erreur personnalisés intégrés dans la charge utile.

HTTP fournit un vocabulaire standardisé que les clients, les proxies et les intermédiaires comprennent déjà.

CATÉGORIES D'ERREURS DANS LES API REST

Les erreurs peuvent être classées en deux grandes catégories :

- **Erreurs côté client (4xx)**
- **Erreurs côté serveur (5xx)**

Une classification correcte est essentielle pour :

- Le débogage
- Les mécanismes de reprise automatique
- La supervision (*monitoring*)
- La sécurité

ERREURS CÔTÉ CLIENT (1/2)

400 Bad Request

Utilisé lorsque :

La requête est syntaxiquement invalide

Des champs requis sont manquants

La validation des entrées échoue

Exemple :

```
{  
  "error": "BadRequest",  
  "message": "Le champ 'email' est requis."  
}
```

401 Unauthorized

Utilisé lorsque :

- Une authentification est requise
- Les informations d'identification sont manquantes ou invalides

Important :

- 401 signifie « **non authentifié** »
- Il doit être accompagné d'instructions d'authentification

403 Forbidden

Utilisé lorsque :

- Le client est authentifié
- Le client n'est pas autorisé à effectuer l'action demandée

Ne pas confondre 401 et 403.

ERREURS CÔTÉ CLIENT (2/2)

404 Not Found

Utilisé lorsque :

- Une ressource n'existe pas
- Le client référence une URI invalide

Choix de conception :

Retourner 404 plutôt que 403 afin d'éviter de révéler l'existence d'une ressource

409 Conflict

Utilisé lorsque :

- Une requête ne peut pas être complétée en raison d'un conflit avec l'état courant

Exemples :

- Création de ressource en double
- Conflits de modifications concurrentes

422 Unprocessable Entity

Utilisé lorsque :

- La requête est syntaxiquement valide
- La sémantique est invalide

Exemple :

Transition d'état invalide

ERREURS CÔTÉ SERVEUR (5XX)

500 Internal Server Error

Indique :

- Une défaillance inattendue côté serveur

Les clients doivent :

- Ne pas relancer aveuglément la requête
- Journaliser l'erreur
- Notifier les opérateurs

503 Service Unavailable

Utilisé lorsque :

- Le serveur est temporairement incapable de traiter la requête
- Une maintenance ou une surcharge est en cours

Peut inclure :

L'en-tête Retry-After

CONCEPTION DES REPRÉSENTATIONS D'ERREUR

La cohérence est essentielle

Les réponses d'erreur doivent :

- Suivre une structure cohérente
- Être documentées
- Être lisibles par machine

Exemple de structure standard :

```
{  
  "error": "ValidationError",  
  "message": "Format d'email invalide",  
  "details": {  
    "field": "email"  
  }  
}
```

Lisibilité humaine vs lisibilité machine

Les réponses d'erreur doivent trouver un équilibre entre :

La lisibilité humaine (pour le débogage)

La lisibilité machine (pour l'automatisation)

Éviter :

Les codes d'erreur cryptiques

Les traces de pile trop verbeuses

GESTION DES ERREURS ET SÉCURITÉ

Les messages d'erreur peuvent involontairement divulguer des informations sensibles.

Bonnes pratiques :

- Ne pas exposer les traces de pile
- Ne pas révéler les identifiants internes
- Éviter les raisons d'échec trop précises pour l'authentification

Exemple :

Préférer « **Identifiants invalides** » à « **Utilisateur inexistant** »

GESTION DES ERREURS ET IDEMPOTENCE

La gestion des erreurs doit respecter l'**idempotence**.

Exemple :

Répéter un PUT ayant échoué ne doit pas produire d'effets de bord

Relancer un POST peut provoquer des doublons s'il n'est pas conçu avec précaution

Des **jetons d'idempotence** peuvent être utilisés pour limiter ce risque.

POURQUOI LE VERSIONNEMENT DES API EST DIFFICILE

Les API sont des **contrats** entre producteurs et consommateurs. Une fois publiées, elles sont difficiles à modifier sans casser les clients.

Les changements cassants incluent :

- La suppression de champs
- Le changement de sémantique des champs
- La modification des URI
- Le changement des mécanismes d'authentification

REST encourage l'évolutivité, mais n'élimine pas la nécessité du versionnement.

TYPES DE CHANGEMENTS D'API

Changements rétrocompatibles

Changements sûrs :

- Ajout de champs optionnels
- Ajout de nouveaux endpoints
- Ajout de nouveaux liens
- Support de nouveaux types de média

Changements cassants

Les changements cassants comprennent:

- Suppression de champs
- Changement de type de données
- Modification de la sémantique des ressources
- Suppression d'endpoints

Les changements cassants nécessitent une gestion explicite des versions.

STRATÉGIES DE VERSIONNEMENT

Versionnement dans l'URI

Exemple :

`/api/v1/students`

Avantages :

- Simple
 - Visible
 - Facile à router
- Inconvénients :
- Viole la stabilité des URI
 - Encourage un versionnement grossier

Versionnement via les en-têtes

Exemple :

`Accept-Version: 2`

Avantages :

- URI plus propres
- Meilleure séparation des préoccupations

Inconvénients :

- Moins visible
- Plus difficile à déboguer

Versionnement par type de média (REST pur)

Exemple :

`Accept: application/vnd.university.student.v2+json`

Avantages :

- Totalement conforme à REST
- Supporte une évolution fine

Inconvénients :

- Plus complexe
- Courbe d'apprentissage plus élevée

DÉPRÉCIATION ET GESTION DU CYCLE DE VIE

Le versionnement ne suffit pas. Les API nécessitent une **gestion de cycle de vie**.

Bonnes pratiques :

- Déprécier avant de supprimer
- Communiquer clairement les calendriers
- Fournir des guides de migration
- Surveiller l'utilisation des anciennes versions

GESTION DES ERREURS EN PRATIQUE : UN EXEMPLE COMPLET

Requête :

POST /students

Réponse en cas d'entrée invalide :

HTTP/1.1 400 Bad Request

Content-Type: application/json

```
{  
  "error": "ValidationError",  
  "message": "L'email est requis"  
}
```

Cette réponse :

- Utilise le code de statut approprié
- Est auto-descriptive
- Peut être exposée en toute sécurité



SÉCURITÉ DANS LES SERVICES WEB RESTFUL

OBJECTIFS PÉDAGOGIQUES DE CE CHAPITRE

Après avoir étudié ce chapitre, les étudiants doivent être capables de :

- Identifier les objectifs de sécurité des API REST
- Expliquer pourquoi HTTPS est obligatoire
- Distinguer authentification et autorisation
- Reconnaître les vulnérabilités courantes des API REST
- Appliquer les bonnes pratiques de sécurité dans la conception des API
- Comprendre le lien entre les contraintes REST et la sécurité

POURQUOI LA SÉCURITÉ EST CRITIQUE DANS LES API REST

Les services web RESTful sont souvent exposés sur Internet et consommés par une grande variété de clients, notamment des navigateurs web, des applications mobiles, des systèmes tiers et des agents automatisés. Cette ouverture rend les API REST puissantes, mais en fait également des cibles privilégiées pour les attaques.

Une erreur fréquente consiste à considérer la sécurité comme un détail d'implémentation ajouté tardivement dans le processus de développement. En réalité, la sécurité doit être **intégrée dès la conception de l'API**. Les choix de conception concernant les URI, les méthodes HTTP, la gestion des erreurs, le caractère sans état et la représentation des données ont tous des implications directes sur la sécurité.

OBJECTIFS DE SÉCURITÉ DANS LES SYSTÈMES RESTFUL

Tout système sécurisé vise à satisfaire les objectifs de sécurité fondamentaux suivants :

Confidentialité

Garantir que les données sensibles ne sont accessibles qu'aux parties autorisées.

Intégrité

Garantir que les données ne sont pas modifiées de manière non autorisée.

Disponibilité

Garantir que le service reste accessible et opérationnel.

Authentification

Vérifier l'identité du client.

Autorisation

Déterminer quelles actions un client authentifié est autorisé à effectuer.

Les API RESTful doivent répondre simultanément à l'ensemble de ces objectifs.

MODÈLE DE MENACE POUR LES API REST

Avant de mettre en œuvre des mécanismes de sécurité, il est essentiel de comprendre le **paysage des menaces**.

Menaces courantes :

- Écoute clandestine du trafic réseau
- Usurpation d'identité
- Accès non autorisé aux ressources
- Attaques par injection
- Attaques par rejeu (*replay attacks*)
- Dénî de service (DoS)
- Fuite d'informations via les messages d'erreur

Une conception sécurisée repose sur un **modèle de menace clair et explicite**.

SÉCURITÉ AU NIVEAU DU TRANSPORT : HTTPS

Pourquoi HTTPS est obligatoire

Les API REST doivent être accessibles **exclusivement via HTTPS**.

HTTPS fournit :

Le chiffrement (confidentialité)

La protection de l'intégrité

L'authentification du serveur

La protection contre les attaques de type *man-in-the-middle*

L'utilisation de HTTP sans TLS :

Expose les identifiants

Divulgue des données sensibles

Rend inefficaces la plupart des autres mécanismes de sécurité

Règle :

➡ Si une API est accessible via HTTP, elle est intrinsèquement non sécurisée.

TLS ne remplace pas la sécurité applicative

HTTPS protège les données en transit, mais :

- N'authentifie pas les utilisateurs
- Autorise les actions
- Ne prévient pas contre les attaques logiques
- Ne valide pas les entrées

TLS constitue une **base nécessaire**, mais pas une solution complète.

MÉCANISMES D'AUTHENTIFICATION COURANTS

a) Authentification basique (Basic Authentication)

Les identifiants sont envoyés avec chaque requête

- Encodés en Base64, mais non chiffrés

Limites :

- Sécurité faible
- Nécessite HTTPS
- Faible scalabilité

L'authentification basique convient uniquement pour :

- Le prototypage
- Les systèmes internes

b) Authentification par jeton (Token-Based Authentication)

Les clients s'authentifient une fois et reçoivent un **jeton**, envoyé ensuite avec chaque requête.

Avantages :

- Sans état
- Scalable
- Adaptée à REST

Les jetons peuvent représenter :

- L'identité de l'utilisateur
- Les permissions
- Une date d'expiration

L'authentification par jeton est l'approche dominante dans les API REST modernes.

AUTORISATION : CONTRÔLE DE L'ACCÈS AUX RESSOURCES

Autorisation au niveau des ressources

Les décisions d'autorisation doivent être prises **au niveau des ressources**.

Exemples :

- Un étudiant peut consulter son propre dossier
- Un administrateur peut supprimer des enregistrements
- Un enseignant peut modifier des notes

L'autorisation doit prendre en compte :

- L'identité de la ressource
- La méthode HTTP
- Le rôle ou les permissions du client

Contrôle d'accès basé sur les rôles et les attributs

RBAC (Role-Based Access Control)

Les permissions sont associées à des rôles.

ABAC (Attribute-Based Access Control)

Les permissions dépendent d'attributs tels que la propriété, le contexte ou le temps.

Les API REST combinent souvent ces deux approches.

VULNÉRABILITÉS COURANTES DANS LES API REST (1/2)

Attaques par injection

Une attaque par injection se produit lorsque des entrées non fiables sont interprétées comme du code.

Exemples :

- Injection SQL
- Injection NoSQL
- Injection de commandes

Mesures de mitigation :

- Validation stricte des entrées
- Requêtes paramétrées
- Éviter la construction dynamique de requêtes

Broken Object Level Authorization (BOLA)

Se produit lorsque :

- Les clients accèdent à des ressources qui ne leur appartiennent pas
- Les identifiants de ressources sont prévisibles

Exemple :

GET /students/43

au lieu de :

GET /students/me

Mesures de mitigation :

- Vérifier systématiquement l'autorisation pour chaque ressource
- Ne jamais faire confiance aux identifiants fournis par le client

VULNÉRABILITÉS COURANTES DANS LES API REST(2/2)

Exposition excessive de données

Se produit lorsque :

- Les API retournent plus de données que nécessaire
- Des champs sensibles sont exposés involontairement

Mesures de mitigation :

- Concevoir soigneusement les représentations
- Utiliser les objets de transfert de données de manière délibérée
- Appliquer le principe du moindre privilège

Affectation massive (Mass Assignment)

Se produit lorsque :

- Les entrées client sont directement mappées sur des objets internes
- Des champs non autorisés sont modifiés

Mesures de mitigation :

- Autoriser explicitement les champs modifiables
- Valider strictement les entrées

SÉCURITÉ ET GESTION DES ERREURS

Une mauvaise conception de la gestion des erreurs peut affaiblir la sécurité.

Bonnes pratiques :

- Éviter les messages détaillés lors des échecs d'authentification
- Ne pas révéler les détails internes du système
- Utiliser des réponses d'erreur cohérentes

Exemple :

Préférer « **Accès refusé** » à « **L'utilisateur existe mais le rôle est insuffisant** »

PROTECTION CONTRE LES ATTAQUES PAR REJEU

Les attaques par rejeu surviennent lorsqu'un attaquant réutilise une requête valide.

Stratégies de mitigation :

- Jetons à durée de vie courte
- Nonces
- Horodatages
- Clés d'idempotence

Ces mécanismes sont particulièrement importants pour les opérations sensibles.

LIMITATION DE DÉBIT ET PROTECTION DE LA DISPONIBILITÉ

Pourquoi la limitation de débit est importante

Les API REST doivent se protéger contre :

- Les abus
- Les attaques par déni de service
- Les surcharges accidentelles

Stratégies de limitation de débit

Approches courantes :

- Nombre de requêtes par seconde et par client
- Limites de rafale (*burst limits*)
- Quotas par clé d'API

Les réponses doivent utiliser :

- 429 Too Many Requests
- Des instructions claires de reprise

JOURNALISATION, SUPERVISION ET AUDIT

La sécurité ne se limite pas à la prévention, mais inclut également la **détection**.

Bonnes pratiques :

- Journaliser les tentatives d'authentification
- Surveiller les comportements inhabituels
- Auditer les opérations sensibles
- Protéger les journaux contre toute altération

Les journaux doivent trouver un équilibre entre :

- Visibilité de la sécurité
- Respect de la vie privée

SÉCURITÉ ET CONTRAINTES REST

Les contraintes REST influencent la conception de la sécurité :

- **Sans état** → authentication par jeton
- **Interface uniforme** → contrôles d'autorisation prévisibles
- **Système en couches** → passerelles API et pare-feu
- **Mise en cache** → éviter la mise en cache de données sensibles

Comprendre REST permet de concevoir des API **sécurisées par construction**.